MACHINE LEARNING

Module Code: CSC83101

LECTURE

TUYISHIMIRE Joseph

tuyishimirejo1992@gmail.com

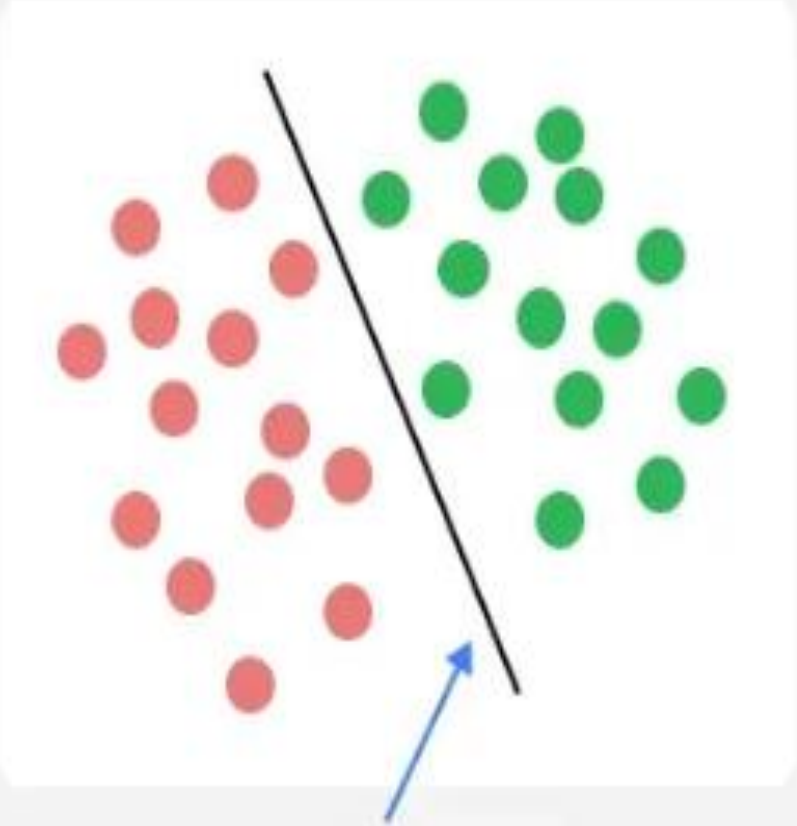
UNIVERSITY Of KIGALI

Support Vector Machine (SVM) Algorithm

Support Vector Machine (SVM) is a supervised machine learning algorithm used for classification and regression tasks. It tries to find the best boundary known as hyperplane that separates different classes in the data. It is useful when you want to do binary classification like spam vs. not spam or cat vs. dog.

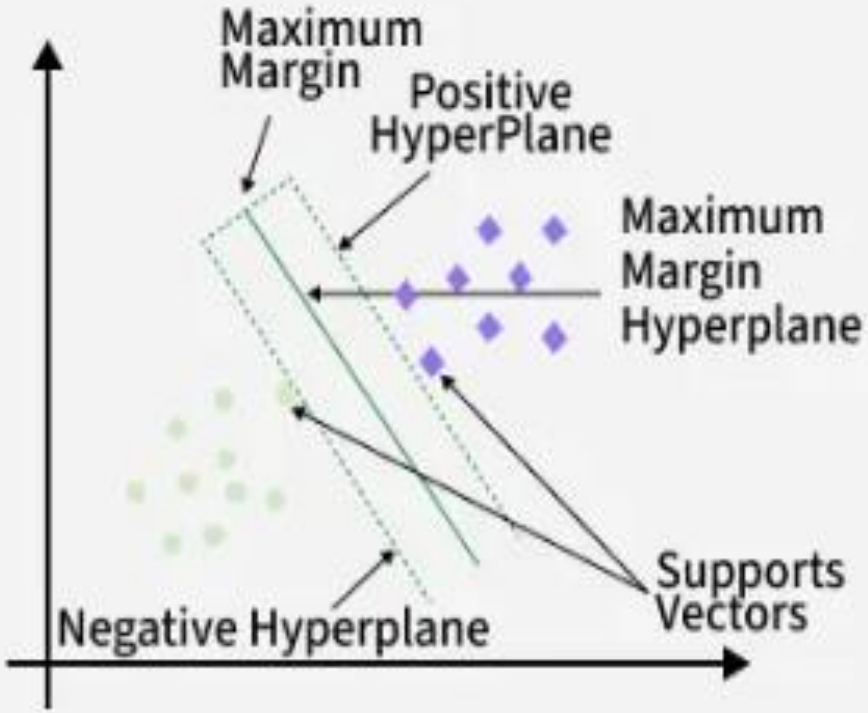
- Support Vector Machine is used for classification and regression.

- It works by finding decision planes that separate data into different classes.

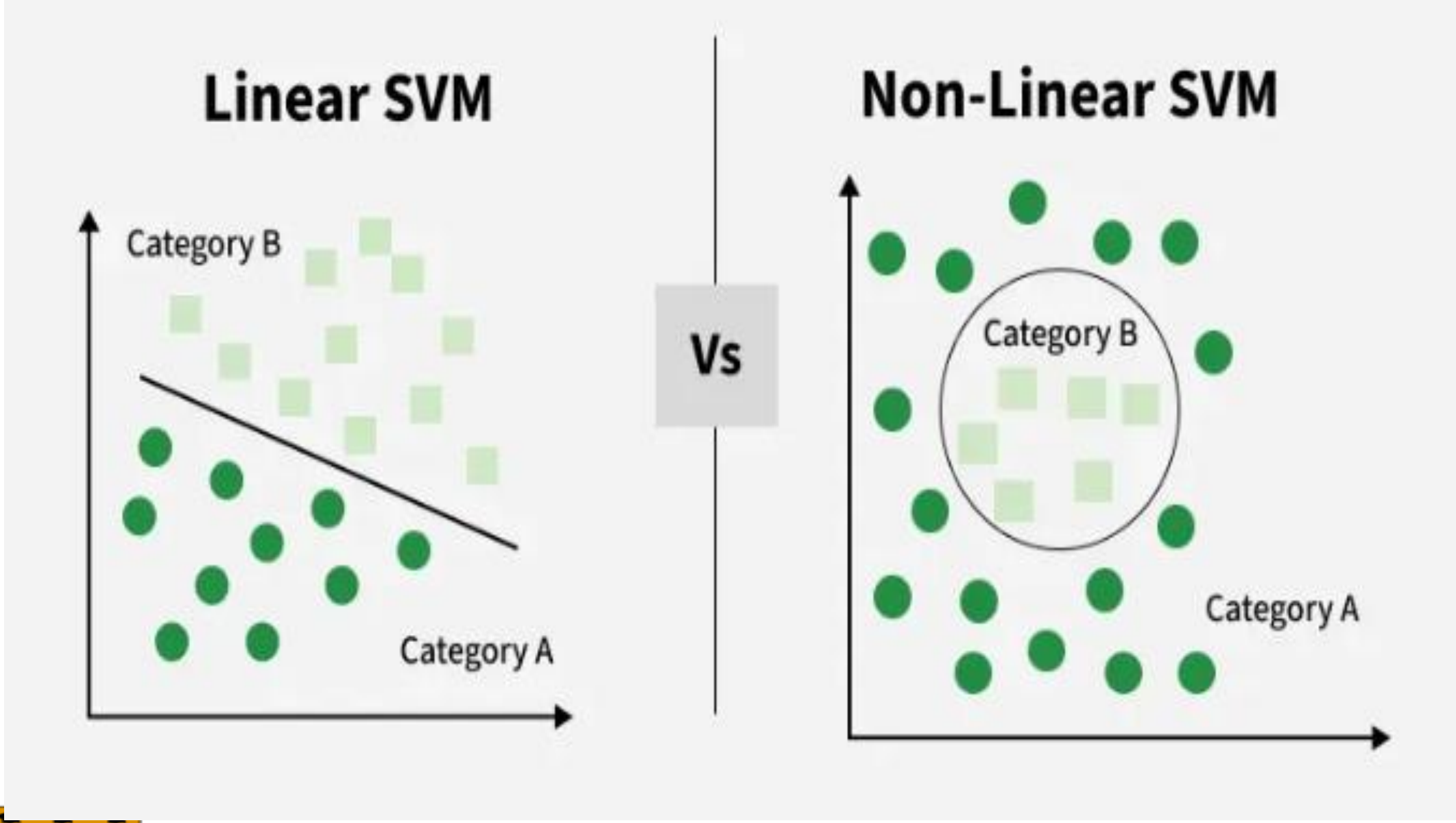


Support Vectors & Hyperplane

- Support Vectors are the closest data points to the hyperplane that define the class boundary.
- A hyperplane is a plane that separates different classes.



UNIVERSITY Of KIGALI



UNIVERSITY Of KIGALI

Key Concepts of Support Vector Machine

- **Hyperplane**: A decision boundary separating different classes in feature space and is represented by the equation $wx + b = 0$ in linear classification.
- **Support Vectors**: The closest data points to the hyperplane, crucial for determining the hyperplane and margin in SVM.
- **Margin**: The distance between the hyperplane and the support vectors. SVM aims to maximize this margin for better classification performance.
- **Kernel**: A function that maps data to a higher-dimensional space enabling SVM to handle non-linearly separable data.

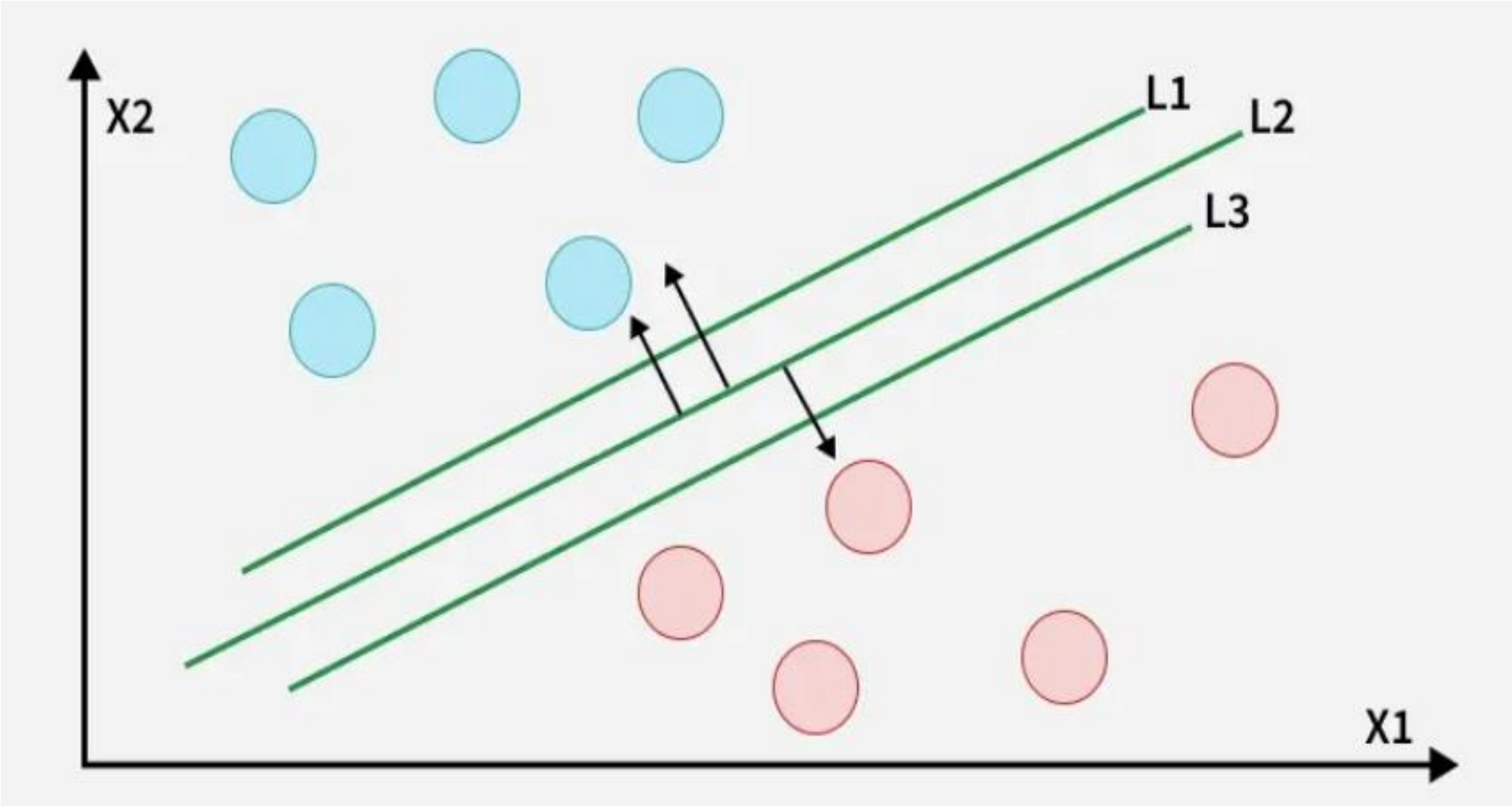
UNIVERSITY Of KIGALI

- **Hard Margin**: A maximum-margin hyperplane that perfectly separates the data without misclassifications.
- **Soft Margin**: Allows some misclassifications by introducing slack variables, balancing margin maximization and misclassification penalties when data is not perfectly separable.
- **C**: A regularization term balancing margin maximization and misclassification penalties. A higher C value forces stricter penalty for misclassifications

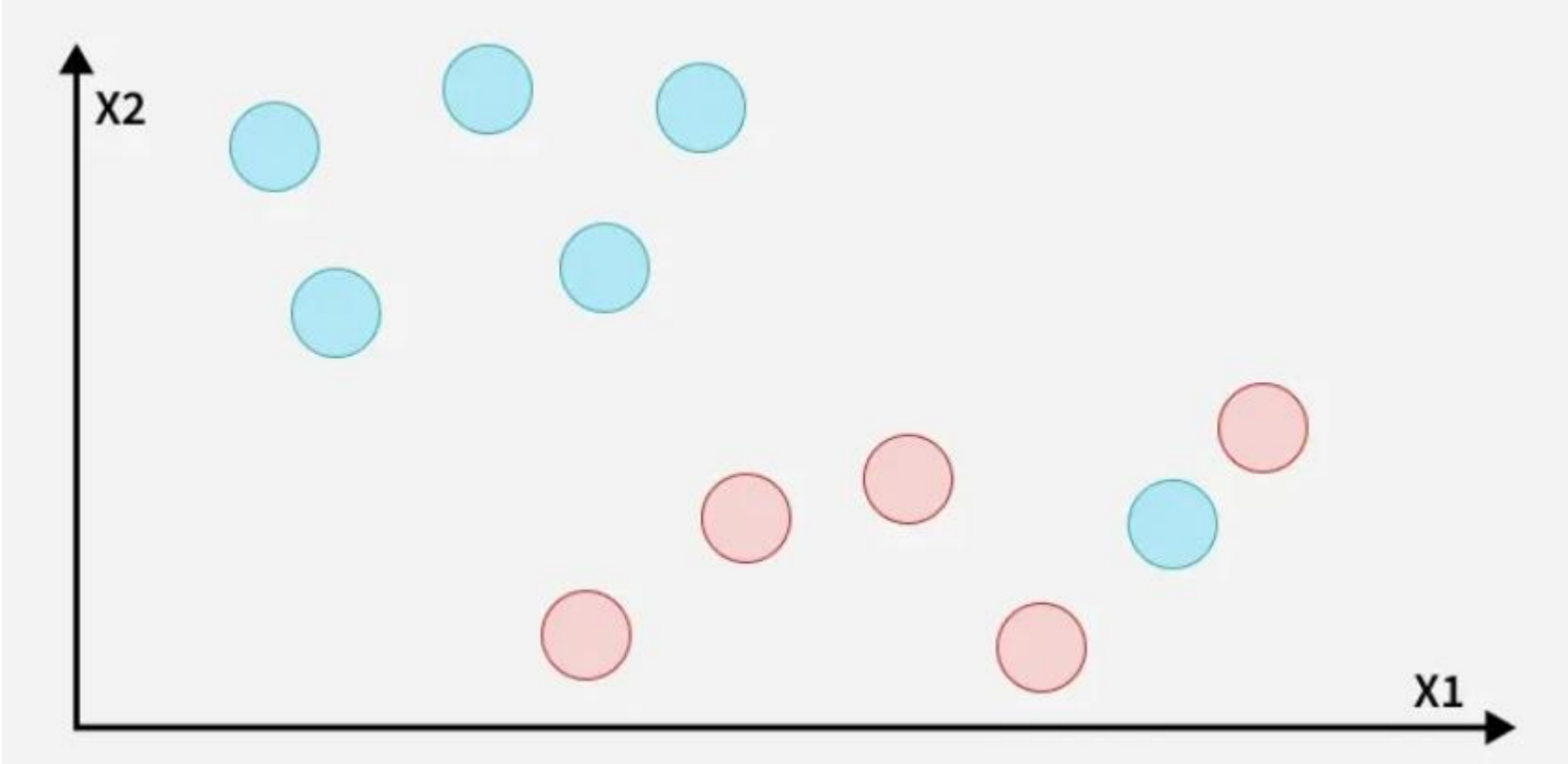
- **Hinge Loss**: A loss function penalizing misclassified points or margin violations and is combined with regularization in SVM.
- **Dual Problem**: Involves solving for Lagrange multipliers associated with support vectors, facilitating the kernel trick and efficient computation.

How does Support Vector Machine Algorithm Work?

The key idea behind the SVM algorithm is to find the hyperplane that best separates two classes by maximizing the margin between them. This margin is the distance from the hyperplane to the nearest data points (support vectors) on each side.


X2
L1
L2
L3
X1

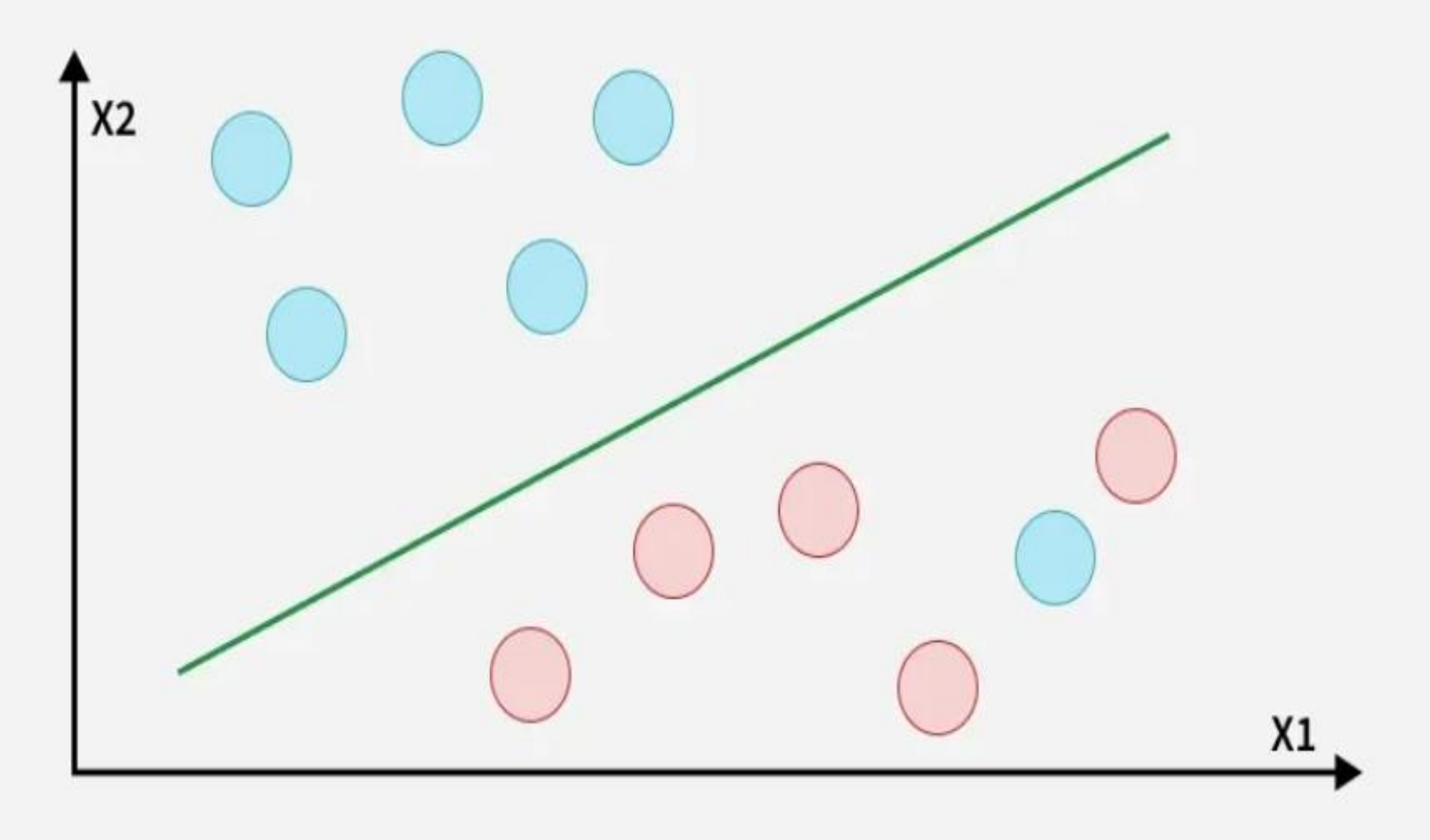
The best hyperplane also known as the **"hard margin"** is the one that maximizes the distance between the hyperplane and the nearest data points from both classes. This ensures a clear separation between the classes. So from the above figure, we choose L2 as hard margin. Let's consider a scenario like shown below:



Here, we have one blue ball in the boundary of the red ball.

How does SVM classify the data?

The blue ball in the boundary of red ones is an outlier of blue balls. The SVM algorithm has the characteristics to ignore the outlier and finds the best hyperplane that maximizes the margin. SVM is robust to outliers.


X2
X1

A soft margin allows for some misclassifications or violations of the margin to improve generalization. The SVM optimizes the following equation to balance margin maximization and penalty minimization:

$$\text{Objective Function} = \left(\frac{1}{\text{margin}}\right) + \lambda \sum \text{penalty}$$

The penalty used for violations is often hinge loss which has the following behavior:

- If a data point is correctly classified and within the margin there is no penalty (loss = 0).
- If a point is incorrectly classified or violates the margin the hinge loss increases proportionally to the distance of the violation.

For two-class classification, given a dataset *(xi, yi)*, where *xi* are feature vectors and *yi* are class labels (either +1 or -1), SVM aims to minimize the function:

$$\min_{w,b} (1/2) ||w||^2$$

Subject to:

$$y_i (w \cdot x_i + b) \geq 1 \text{ for all } i$$

where:

- **w** represents the weight vector,
- **b** is the bias term,
- $||\mathbf{w}||^2$ ensures the maximization of the margin.

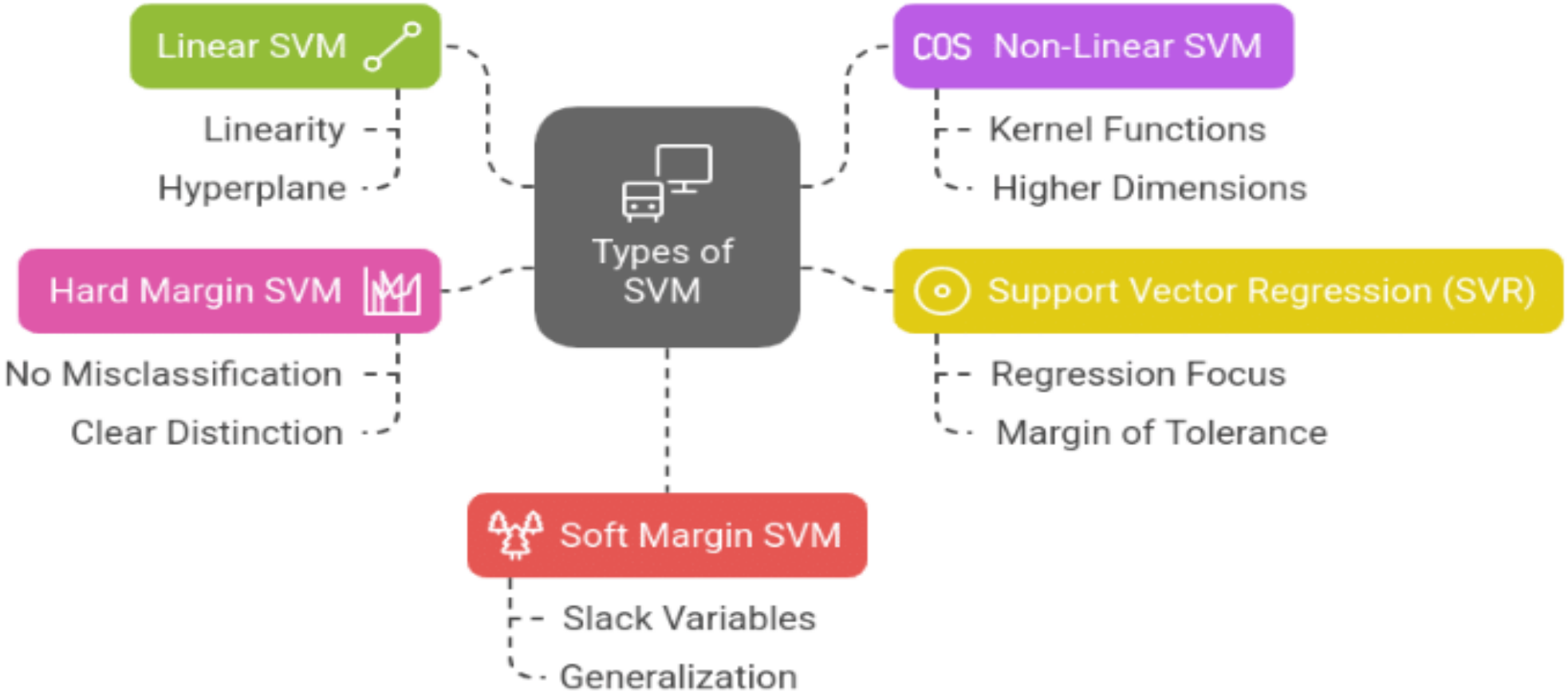
$$\min_{w,b,\xi} (1/2) ||w||^2 + C \Sigma \xi_i$$

Here, **C** controls the trade-off between maximizing the margin and minimizing classification errors.

Types of SVM

SVM can be classified into different types based on the nature of the dataset:

Types of Support Vector Machines (SVM)



UNIVERSITY of KIGALI

- **Linear SVM**: Used when data can be separated using a straight hyperplane. It is suitable for datasets where classes are linearly separable. The decision boundary is a straight line (in 2D) or a plane (in higher dimensions).

- **Non-Linear SVM**: When data is not linearly separable, kernel functions are used to map data into a higher-dimensional space where separation is possible. Kernel functions like RBF and polynomials are commonly used in non-linear SVM.

- **Support Vector Regression (SVR)**: A variation of SVM used for regression problems instead of classification. It works similarly to classification SVM but tries to fit a function within a margin of tolerance rather than finding a strict boundary between categories.

- **Hard Margin SVM**: Assumes that the data is perfectly separable and aims to find a hyperplane that classifies all data points correctly with no tolerance for misclassification. This works well when there is a clear distinction between classes

Soft Margin SVM: Introduces slack variables to handle cases where classes may overlap slightly. It allows some misclassifications to improve generalization and prevent overfitting

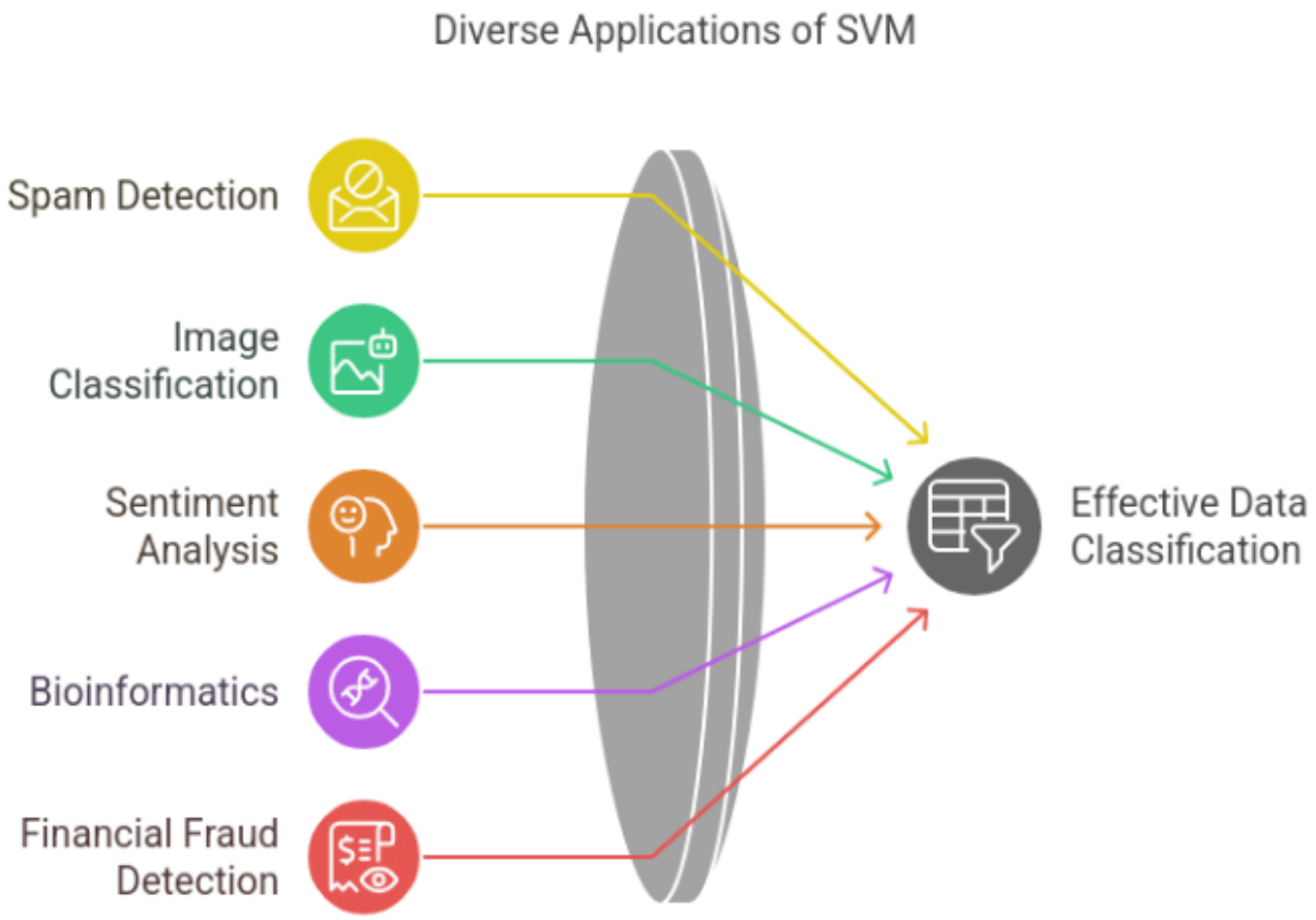
The Kernel Trick: Handling Non-Linearity

- Real-world data is often non-linearly separable. SVM uses **kernel functions** to transform data into a higher-dimensional space where it becomes separable.

Some popular kernels include:

- **Linear Kernel:** $K(x, y) = x \cdot y$ (used in *linear support vector machine*)
- **Polynomial Kernel:** $K(x, y) = (x \cdot y + c)^d$
- **Radial Basis Function (RBF) Kernel:** $K(x, y) = e^{-\gamma\|x - y\|^2}$

Real-World Applications of SVM



UNIVERSITY of KIGALI

General SVM Workflow in Python with scikit-learn:

1. Import necessary libraries
2. Load and prepare the data
3. Encode categorical variables (e.g., one-hot encoding)
4. Split the data into training and testing sets
5. Create and train the SVM model
6. Make predictions on the test set
7. Evaluate model performance

Explanation of Key Concepts:

1. Support Vector Machine (SVM): A supervised learning algorithm used for classification or regression tasks. It finds the hyperplane that best separates classes.
2. One-Hot Encoding: Converts categorical variables (like Contract type) into a set of binary columns so the model can understand them.
3. Train/Test Split: Divides data into training (to learn patterns) and testing (to evaluate performance) sets.
4. Kernel: SVM uses kernels (e.g., linear, RBF) to transform data for better separation.

Step-by-Step Guide to Implement SVM

- **Step 1: Import Required Libraries**

Before starting, import the necessary libraries for data handling, preprocessing, model building, and evaluation

Libraries:

- import pandas as pd
- from sklearn.model_selection import train_test_split
- from sklearn.preprocessing import OneHotEncoder
- from sklearn.compose import ColumnTransformer
- from sklearn.pipeline import Pipeline
- from sklearn.svm import SVC
- from sklearn.metrics import accuracy_score, classification_report

1. pandas

- Purpose: pandas is a powerful library for data manipulation and analysis in Python.
- Use case: It provides data structures like DataFrames and Series, which make it easy to load, clean, explore, and manipulate structured data.
- In your code: You use pandas to create and manage the dataset in a tabular format.

2. sklearn.modelselection.traintest_split

- Purpose: This function is used to split your dataset into training and testing subsets.
- Use case: It helps evaluate the model's generalization ability by training it on one part of the data and testing it on unseen data.
- In your code: You use it to divide your features and labels into training and testing sets with a specified test size.

sklearn.preprocessing.OneHotEncoder

- Purpose: This transformer converts categorical variables into a format that can be provided to ML algorithms to do a better job in prediction.
- Use case: It converts each category value into a new binary column (1 or 0).
- In your code: You use OneHotEncoder to transform the categorical feature Contract into numerical binary features.

sklearn.compose.ColumnTransformer

- Purpose: This allows selective application of transformers to columns of a dataset.
- Use case: You can apply different preprocessing steps to different columns (e.g., one-hot encode categorical columns, scale numeric columns).
- In your code: You apply one-hot encoding only to the categorical column(s), while leaving other columns unchanged.

sklearn.pipeline.Pipeline

- Purpose: Pipeline helps to chain multiple steps (transformations and estimator training) into a single object.
- Use case: It ensures that the same preprocessing applied during training is also applied during prediction, making code cleaner and less error-prone.
- In your code: You combine the preprocessing step (one-h

sklearn.svm.SVC

- Purpose: SVC stands for Support Vector Classifier, an implementation of the Support Vector Machine algorithm for classification tasks.
- Use case: SVM tries to find the best boundary (hyperplane) that separates classes with the maximum margin. It works well on both linear and non-linear separable data (using kernels like RBF).
- In your code: You use SVC as the classification model to predict whether a customer will churn or not.

sklearn.metrics.accuracy_score

- Purpose: This function computes the accuracy classification score, which is the fraction of correctly predicted samples.
- Use case: It is a simple and intuitive metric for classification model evaluation.
- In your code: You use it to evaluate how many churn predictions your model got right on the test set.

sklearn.metrics.classification_report

- Purpose: This function builds a text report showing the main classification metrics such as precision, recall, F1-score, and support for each class.
- Use case: It provides a detailed performance overview, especially useful for imbalanced classes.
- In your code: You use it to get a comprehensive evaluation of your model's performance beyond just accuracy.

Step 2: Load and Prepare Your Dataset

```
# Seed for reproducibility
np.random.seed(42)
# Generate dataset
X, y = make_classification(
    n_samples=100,
    n_features=2,
    n_redundant=0,
    n_clusters_per_class=1,
    class_sep=2.0,
    random_state=42
)
# Create DataFrame
df = pd.DataFrame(X, columns=['Feature_1', 'Feature_2'])
df['Target'] = y
```

```
# Introduce missing values in Feature_1
n_missing = 10  # 10 missing values
missing_indices = np.random.choice(df.index, n_missing,
replace=False)
df.loc[missing_indices, 'Feature_1'] = np.nan
print("Dataset with missing values in Feature_1:")
print(df.loc[missing_indices])
print("\nCount of missing values per column:")
print(df.isnull().sum())
```

Impute missing values with mean

Handle Missing Values with Imputation

General Syntax

```
df['feature_column'] =
df['feature_column'].fillna(df['feature_column'].mean())

df['Feature_1'] =
df['Feature_1'].fillna(df['Feature_1'].mean())
```

Step 2: traintestsplit from sklearn.model_selection

Purpose: Split dataset into training and testing sets.

General Syntax:

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(X, y,
                                                    test_size=0.2,
                                                    random_state=42)
```

3.Separate Features and Target

General Syntax:

X = df[feature_columns]

y = df[target_column]

Extract features and target variables separately for model training.

```
X = df[['Feature_1', 'Feature_2']]
y = df['Target']
```

Split Dataset into Training and Testing Sets

General Syntax:

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size, random_state
)
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

Explanation:

- Splits dataset into training and testing subsets.
- test_size=0.2 means 20% data for testing.
- random_state for reproducibility.

Train SVM Classifier

- **General Syntax:**

model = SVC(kernel, random_state)

model.fit(X_train, y_train)

- **Explanation:**

Initialize SVM with specified kernel (linear here).

Train model on training data.

- svm = SVC(kernel='linear', random_state=42)
- svm.fit(X_train, y_train)

Predict and Evaluate Model (with Confusion Matrix)

- **General Syntax:**

✓ **y_pred = model.predict(X_test)**

✓ **print(accuracy_score(y_test, y_pred))**

✓ **print(classification_report(y_test, y_pred))**

✓ **cm = confusion_matrix(y_test, y_pred)**

Print(cm)

Optional: visualize confusion matrix using seaborn heatmap or matplotlib

```
# Calculate accuracy
accuracy = accuracy_score(y_test, y_pred)
print(f"Accuracy: {accuracy:.4f}")
# Generate confusion matrix
cm = confusion_matrix(y_test, y_pred)
print("Confusion Matrix:")
print(cm)
# Generate detailed classification report (precision, recall, f1-score)
report = classification_report(y_test,y_pred)print("Classification Report:")
print(report)
```

Step 8: Plot Decision Boundary and Data Points (for 2D features)

```
import matplotlib.pyplot as plt
import numpy as np
# Select two features for visualization, e.g., first two
columns of X_test_scaledX_vis = X_test_scaled[:, :2]
# Create a mesh grid over the feature space
x_min, x_max = X_vis[:, 0].min() - 1, X_vis[:, 0].max() + 1
y_min, y_max = X_vis[:, 1].min() - 1, X_vis[:, 1].max() + 1
xx, yy = np.meshgrid(np.arange(x_min, x_max, 0.02),
np.arange(y_min, y_max, 0.02))
```

```
# Predict on each point in the grid
Z = svm_model.predict(np.c_[xx.ravel(), yy.ravel()])
Z = Z.reshape(xx.shape)
# Plot the decision boundary by assigning a color to each point in the mesh
plt.contourf(xx, yy, Z, alpha=0.3)
# Scatter plot of the actual test points colored by their true label
plt.scatter(X_vis[:, 0], X_vis[:, 1], c=y_test, edgecolors='k', cmap=plt.cm.coolwarm, s=50)
plt.xlabel('Feature 1 (scaled)')
plt.ylabel('Feature 2 (scaled)')
plt.title('SVM Decision Boundary and Test Data')
plt.show()
```

Notes:

- This visualization only works well with **two features**. If your dataset has more features, select two relevant ones for plotting.
- The plot shows regions classified as each class and overlays the actual test points colored by their true label, allowing you to see how well the SVM separates the data.

EXAMPLE 1 /Scenario Development:

A telecom company is experiencing customer losses due to churn when customers discontinue their service subscriptions. To better understand and reduce churn, the company wants to predict whether a customer is likely to leave based on specific features such as:

- Monthly Charges: The amount the customer pays every month.
- Tenure: The number of months the customer has been with the company.
- Contract Type: The nature of the customer's contract (e.g., month-to-month, one year, two years).

The company has collected historical data where each customer's status is labeled as **Churn = 1** if they left the service or **Churn = 0** if they stayed.

Using this data, they plan to build a predictive model that can identify customers at high risk of churning, allowing targeted retention efforts such as personalized offers or improved customer support.

Problem Identification:

Type of Problem: This is a binary classification problem where the goal is to classify customers into two categories churners (1) and non-churners (0).

- Objective: Develop a predictive model that accurately forecasts customer churn based on input features.
- Challenges:
- Handling diverse data types (numerical and categorical).
- Managing imbalanced data if churners are a minority.
- Ensuring the model generalizes well to new customers.
- Using the model outputs to inform business strategies for customer retention.

UNIVERSITY Of KIGALI

```python
#Step 1: Create the dataset
import pandas as pd
import numpy as np

# Sample dataset
data = {
    'MonthlyCharges': [70, 90, 45, 60, 80, 85, 50, 65, 95, 55],
    'Tenure': [12, 24, 5, 10, 18, 22, 4, 8, 30, 6],
    'Contract': ['Month-to-month', 'One year', 'Month-to-month', 'Two year', 'One year',
                 'Two year', 'Month-to-month', 'Two year', 'One year', 'Month-to-month'],
    'Churn': [1, 0, 1, 0, 0, 0, 1, 0, 0, 1]
}

df = pd.DataFrame(data)
print(df)
```

	MonthlyCharges	Tenure	Contract	Churn
0	70	12	Month-to-month	1
1	90	24	One year	0
2	45	5	Month-to-month	1
3	60	10	Two year	0
4	80	18	One year	0
5	85	22	Two year	0
6	50	4	Month-to-month	1
7	65	8	Two year	0
8	95	30	One year	0
9	55	6	Month-to-month	1

```python
"""Step 2: Preprocess the data
Encode categorical variables (Contract)
Separate features and target
Split into train and test sets
Scale features (important for SVM)"""
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

# One-hot encode Contract
df_encoded = pd.get_dummies(df, columns=['Contract'])

# Features and target
X = df_encoded.drop('Churn', axis=1)
y = df_encoded['Churn']

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

# Scale features
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

```python
#Step 3: Train the SVM classifier
from sklearn.svm import SVC


svm_model = SVC(kernel='linear', random_state=42)
svm_model.fit(X_train_scaled, y_train)
```

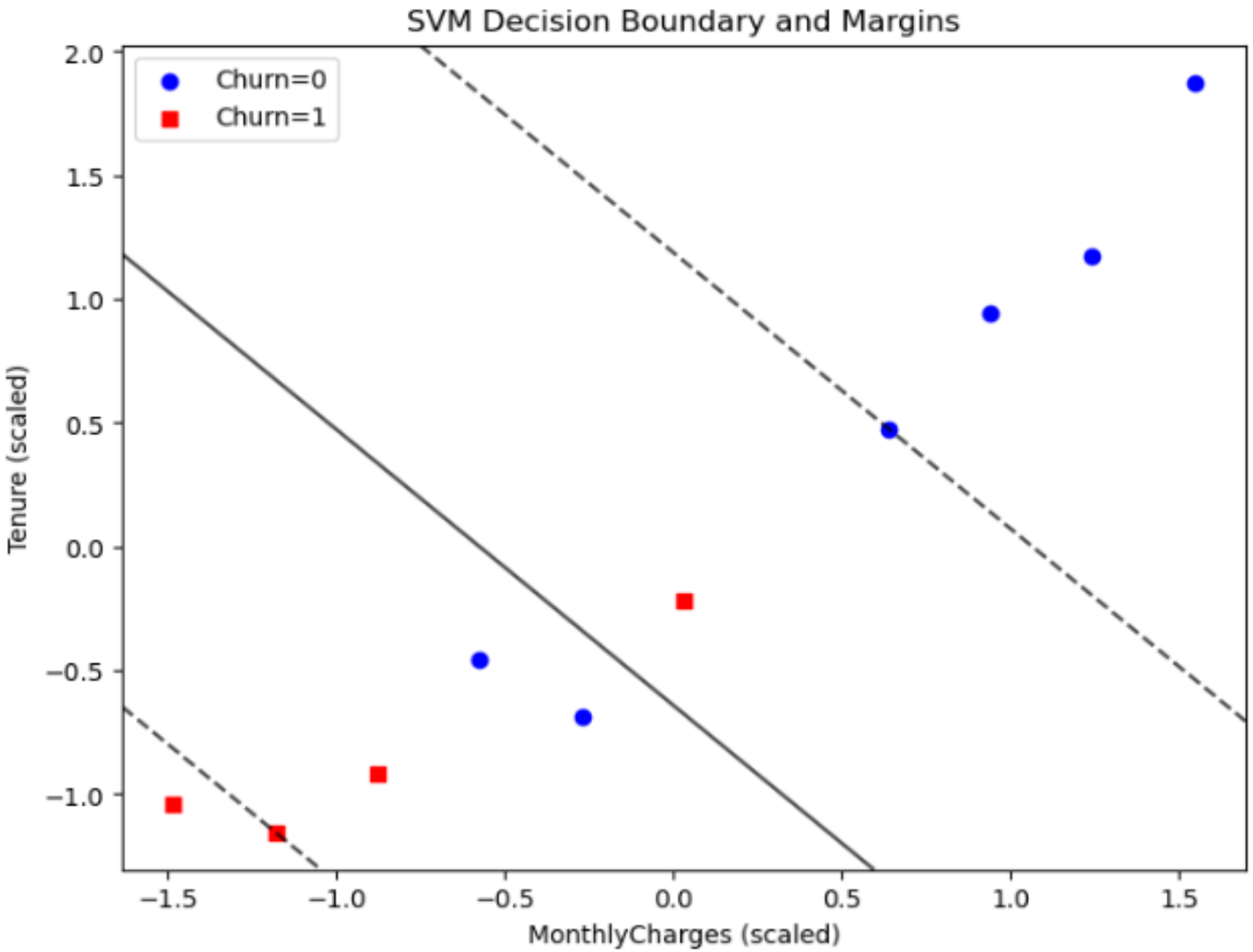
```
SVC
SVC(kernel='linear', random_state=42)
```

```python
#Step 4: Predict on test data
y_pred = svm_model.predict(X_test_scaled)
```

```python
#Step 5: Evaluate model performance
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

( accuracy_score(y_test, y_pred))
(confusion_matrix(y_test, y_pred))
( classification_report(y_test, y_pred))
print(f"\nAccuracy: {accuracy:.2f}")
print("\nConfusion Matrix:\n", conf_mat)
print("\nClassification Report:\n", class_report)
```

```
Accuracy: 0.33

Confusion Matrix:
 [[1 0]
 [2 0]]

Classification Report:
              precision    recall  f1-score   support

           0       0.33      1.00      0.50         1
           1       0.00      0.00      0.00         2

    accuracy                           0.33         3
   macro avg       0.17      0.50      0.25         3
weighted avg       0.11      0.33      0.17         3
```

```python
# Plotting
plt.figure(figsize=(8,6))

# Plot points
for label, marker, color in zip([0, 1], ['o', 's'], ['blue', 'red']):
    plt.scatter(X_2d_scaled[y_2d == label, 0], X_2d_scaled[y_2d == label, 1],
                marker=marker, color=color, label=f"Churn={label}")

# Plot the separating hyperplane
ax = plt.gca()
xlim = ax.get_xlim()
ylim = ax.get_ylim()

# Create grid to evaluate model
xx = np.linspace(xlim[0], xlim[1], 30)
yy = np.linspace(ylim[0], ylim[1], 30)
YY, XX = np.meshgrid(yy, xx)
xy = np.vstack([XX.ravel(), YY.ravel()]).T
Z = svm_2d.decision_function(xy).reshape(XX.shape)

# Plot decision boundary and margins
ax.contour(XX, YY, Z, colors='k', levels=[-1, 0, 1], alpha=0.7,
           linestyles=['--', '-', '--'])

plt.xlabel('MonthlyCharges (scaled)')
plt.ylabel('Tenure (scaled)')
plt.title('SVM Decision Boundary and Margins')
plt.legend()
plt.show()
```


SVM Decision Boundary and Margins
Churn=0
Churn=1
Tenure (scaled)
MonthlyCharges (scaled)
2.0
1.5
1.0
0.5
0.0
−0.5
−1.0
−1.5
−1.0
−0.5
0.0
0.5
1.0
1.5